

SD	Distributed Systems
26/27	Sockets, Event Streaming, Queues and modularity.
	Water Management Network

Preamble

The goal of this practical is for students to extend and consolidate their knowledge of basic socket communication technology, studied during the theory sessions.

Sockets are the foundation of communication between computers and represent the basic link point between our application and the Internet, using very few network resources. As a trade-off, applications that use sockets as a communication mechanism must interpret the messages they exchange with each other, meaning they must establish a protocol or communication agreement between them. This need implies strong coupling between distributed applications that use sockets as a communication mechanism, since all parties must know in advance the structure of the messages that will be exchanged and encode it in their programming.

On the other hand, other current technologies and architectures oriented toward real-time event streaming, based on sockets but at a higher level in terms of construction and development, allow for the implementation of less coupled and more resilient solutions than these.

This practical establishes the initial framework for a development that will be expanded throughout the semester and that will allow the student to create an application similar to those developed today in enterprise environments, focusing on the use and integration of different communication paradigms that can be used.

Smart irrigation in cities

Efficient water management has become one of the major challenges facing today's cities. Maintaining parks, gardens, and urban green areas requires constant, well-planned irrigation, but this consumption must be reconciled with the increasingly pressing need for rational and sustainable water use.

Manual or unsupervised irrigation presents significant inefficiencies: over-watering in areas with sufficient moisture, undetected leaks that go on for hours or days without anyone noticing, or out-of-service stations that go unnoticed until maintenance staff physically visit them. In contrast, a network of centrally connected and supervised irrigation stations makes it possible to know the status of every point in the city at all times, detect incidents early, avoid wasting a scarce resource, and facilitate the planning of green-area maintenance.

In addition, continuous monitoring of moisture and consumption provides valuable information for technical, economic, and environmental decision-making, and lays the groundwork for future irrigation optimization based on real data from each area.

This is not a theoretical problem: in Spain, actual water losses in distribution networks (leaks, breaks, and faults) are estimated at 14.6% of the total volume supplied (INE, 2024). Alicante, by contrast, sits well above that average, with a technical hydraulic efficiency of 88.55%, supported by the digitalization of its sensor and meter network. The city itself also has a reclaimed-water network that provides irrigation access to nearly 87% of its green areas, thereby reducing the consumption of drinking water for this purpose.

In this context, the proposed practical seeks to bring the student closer to the challenge of implementing the theoretical concepts of the distributed systems course in a real-world problem: managing and supervising, in a comprehensive, resilient, scalable, and secure manner, an irrigation network covering multiple parks and gardens across a city.

Specification

The goal of the practical is to develop a distributed system that implements a simulation of a solution for the centralized management of irrigation for a city's parks and gardens.

Functional description

Students must implement a real-time online control solution called "WaterManagement".

Scope

The solution's goal is to build a system that simulates the management of a network of irrigation stations (hereinafter also **"Watering Station"** or **WS**) distributed across a city's parks and gardens, and its use both by the city council/network operator and by the maintenance staff who activate and supervise irrigation on site.

Our simulation will consist of, starting from a map of a fictitious city, having a series of irrigation stations distributed among its various parks and gardens, where operators, equipped with an application, will be able to activate and supervise irrigation in each area.

All irrigation stations are managed from a central system that will register them, operate them, and monitor them in the event of incidents.

To build the whole system, we will rely on a series of distributed components that will be interconnected and will form the solution in a resilient, scalable, and secure manner.

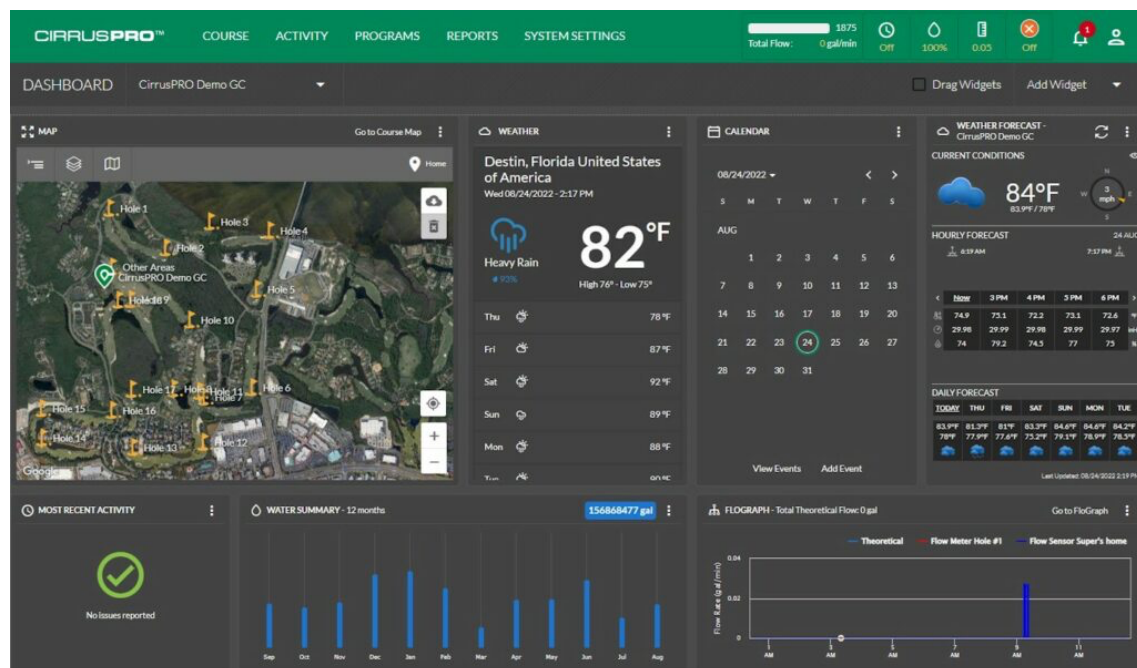


Image 1: Irrigation systems from the company Rain Bird

CENTRAL. Network monitoring system

We will have a dashboard or monitoring panel where the real-time operating details of the entire system can be observed, including the irrigation stations, activation requests, and the status of the central system.

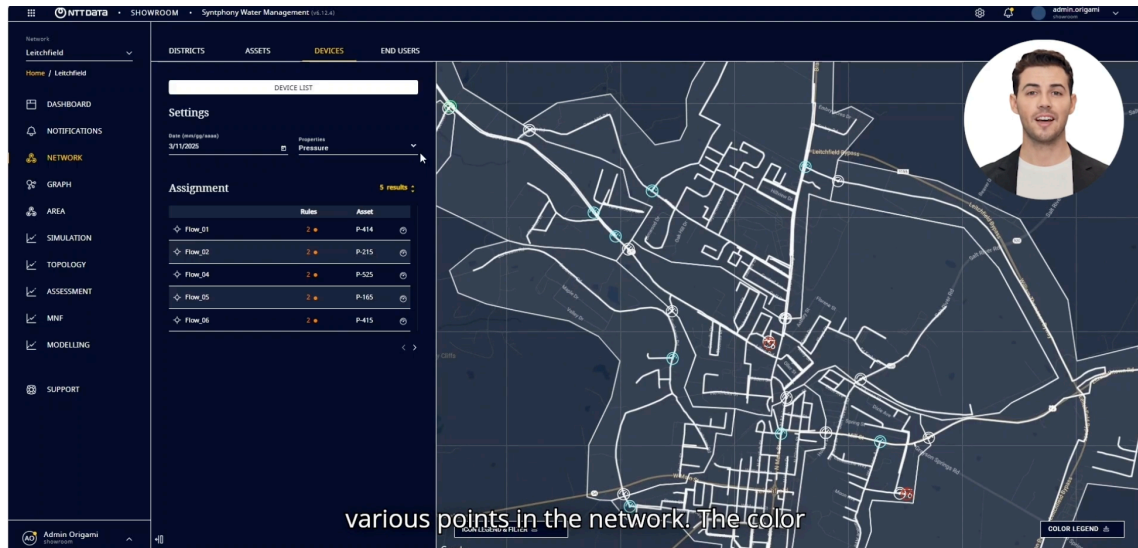


Image 2: Example of a monitoring system for a water network

On the panel showing detailed information, the information to be displayed will be as follows:

- ID of the irrigation station.
- Location of the station (park/garden/city area).
- Amount of water being supplied.
- Status:
 - **Available (activated):** The WS is functioning correctly and is waiting for an irrigation request. It will be displayed with a **GREEN** background.
 - **Watering:** The WS is functioning correctly and is, at that moment, supplying irrigation to the area it covers. It will be displayed with a **GREEN BLINKING** background, with the following data displayed:
 - Real-time flow rate in liters/minute.
 - Real-time accumulated volume in liters.
 - ID of the operator who activated the irrigation.
 - **Leak:** The WS is connected to the central system, but has a leak detected by its sensor, and irrigation must not occur until the fault is resolved. It will be displayed with a **RED** background.
 - **Out of service (stopped):** The WS is functioning correctly, but cannot be used deliberately by order of CENTRAL. It will be displayed with a **ORANGE** background with the caption "Out of Service".
 - **Disconnected:** The WS is not connected to the central system (due to a fault, network failure, or any other reason) and its use cannot be monitored. It will be displayed with a **GRAY** background.

FIELD OPERATORS (FO)

Operators (parks and gardens maintenance staff) will have an application that allows them to activate irrigation at a given station. This application will send CENTRAL the activation request, and CENTRAL will respond by authorizing or denying it.

The central system, as will be seen in the solution mechanics section, must ensure that the irrigation station is available before authorizing activation.

IRRIGATION STATIONS (WATERING STATION - WS)

Stations located in the various parks and gardens of the city where irrigation of the green area they cover is carried out.

Each irrigation station includes:

- **Flow meter:** measures the amount of water being supplied.
- **Leak detector:** detects abnormal water losses in the station's installation.
- **Solenoid valve actuator:** allows opening or closing the water flow to start or stop irrigation.

The states a WS can enter are those described earlier in the CENTRAL control panel.

Irrigation stations support the following functionalities:

- Register with CENTRAL: to do so, they will send their ID and location to CENTRAL.
- Activate or stop.
- Water: provide the irrigation service in two ways:
 - Manually, via an option on the station itself.
 - Through a request from the operator's application (via CENTRAL, as will be seen later) or even from CENTRAL itself.
- Measure the volume of water being supplied.

These stations also have a continuous self-monitoring capability (sensors and actuator) that makes it possible to check their proper functioning or detect a possible fault.

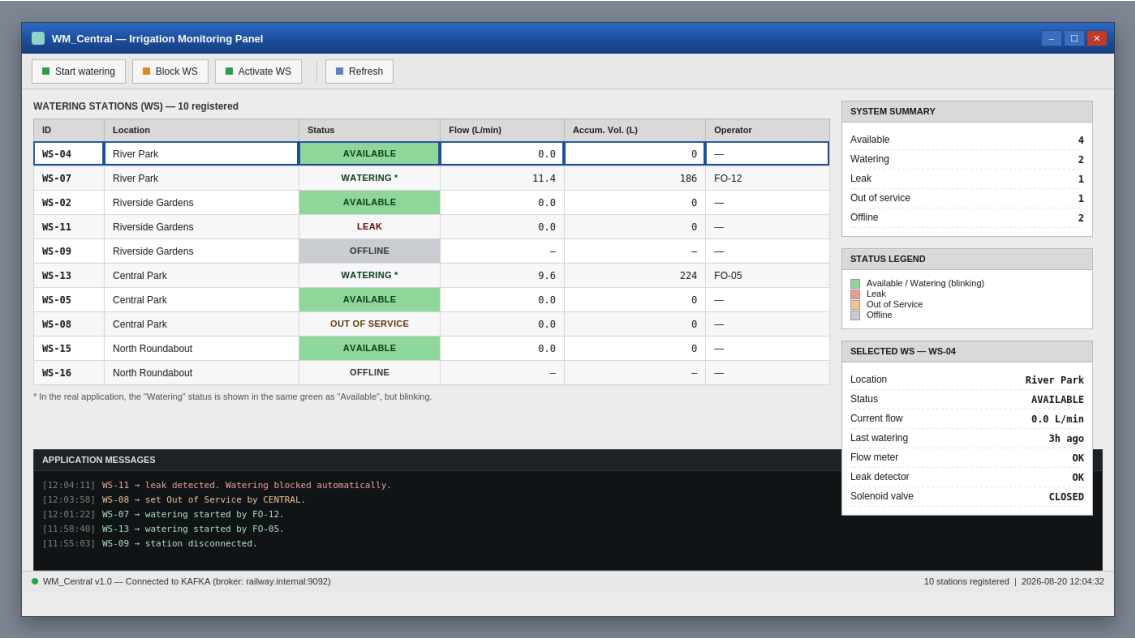


Figure 1: Example of the central system's control panel in desktop application version.

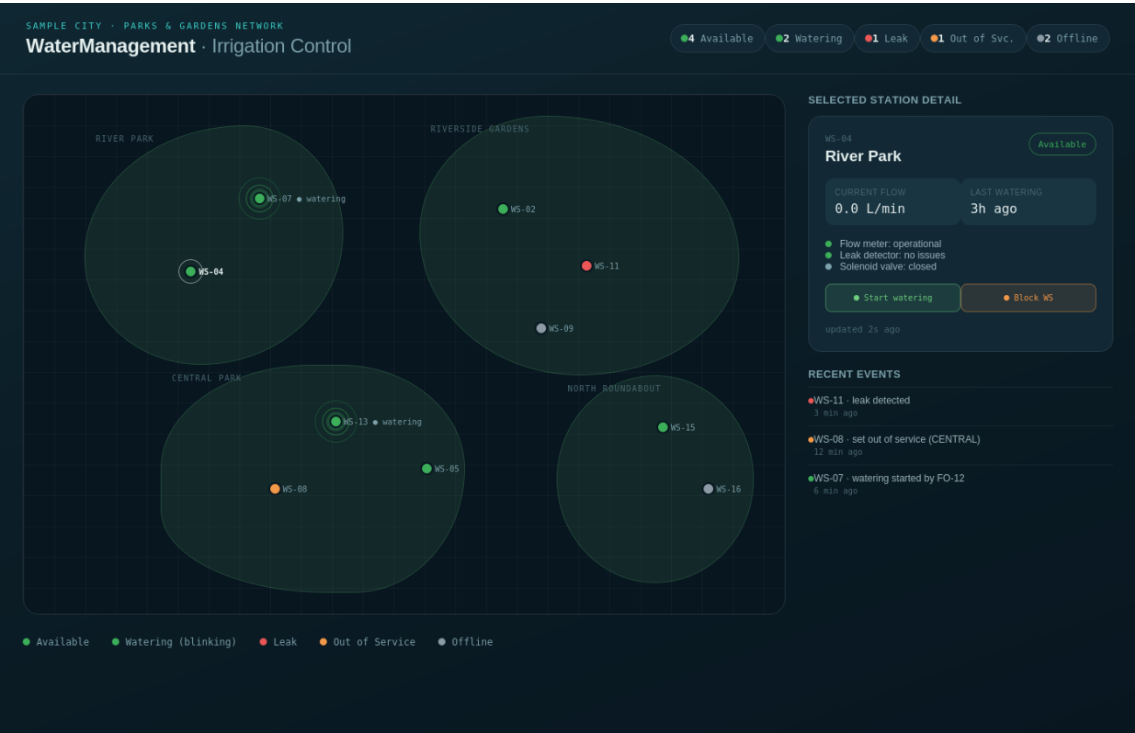


Figure 2: Example of the central system's control panel in web front-end version.

Solution mechanics

The system starts by running the central module (hereinafter "CENTRAL"), which will keep running indefinitely without ever shutting down (unless an exception or sudden lockup causes an uncontrolled shutdown).

The following sequence then takes place:

1. On any startup or restart, CENTRAL will check (in its DB) whether it already has irrigation stations registered (with their location) and will display them on its monitoring and control panel in their corresponding state. **IMPORTANT:** until an irrigation station connects to CENTRAL, CENTRAL cannot know the station's actual status. In that case, it will show it with the DISCONNECTED status.

2. CENTRAL will always be waiting to:

- a. Receive registration and enrollment requests for a new irrigation station.
- b. Receive irrigation activation requests.

3. Operators, from their application or from a menu on the station itself, will request irrigation activation at any station. For the academic purpose of automating system tests, in addition to being able to request activation manually, the operator's application will also be able to read the activations to request from a file with the following format

4. CENTRAL will carry out the appropriate checks to validate that the irrigation station is available (activated, with no fault or leak detected) and, if applicable, will request authorization from the station to proceed with irrigation. The whole process will require notifying the operator of the steps taking place until the irrigation is authorized or denied. These messages must be clearly displayed on screen, both in the operator's application and in CENTRAL.

5. Once registered and connected to CENTRAL, irrigation stations will remain in a resting state, waiting for an operator to request an irrigation activation, either through the station's own interface or through their application.

6. Once CENTRAL has completed all checks and sent the notification to both the WS and the operator's application to proceed with activation, the system will proceed to open the solenoid valve.

7. During irrigation, the station will be continuously sending information (every second) to CENTRAL, which will display on its monitoring panel the flow rate and accumulated volume taking place at each WS, as indicated in earlier sections. The operator's application will likewise display those states of the WS that is watering.

8. **Irrigation will continue indefinitely** until one of these two conditions occurs:

- A preset time limit set when irrigation started is reached.
- A menu option is executed on the WS to end the supply.

In both cases, the WS will notify CENTRAL, which will in turn send a final irrigation summary (total volume consumed, duration, etc.) to the operator. The WS will return to the resting state, waiting for a new activation request.

9. Throughout the system's operation, the irrigation station will be checking the health status of its own elements. In the event of a fault (leak detection), it will notify CENTRAL of this situation.

Once the contingency has been resolved, a new message will be sent to CENTRAL and the WS will become available again. If the fault occurs during an ongoing irrigation, it must end immediately, informing all modules and applications of the situation. All information must be clearly visible on screen, both in CENTRAL and on the station's monitor.

10. CENTRAL will receive all statuses and irrigation events from all WS. Operators will only receive messages from the specific WS serving them, but they will also be able to see all WS that are available.

11. CENTRAL will always remain running, waiting for new activation requests. It will additionally have options to arbitrarily send any of the following commands to one or all WS:

- **Start irrigation:** It will behave the same way as if an operator had done it.
- **Block the WS:** The WS will end any irrigation in progress and set its state to ORANGE, showing the message "Out of Service".
- **Activate the WS:** The WS will return to the ACTIVATED state and set its color to GREEN.

The action unfolds asynchronously, that is, without "turns", so that each operator, CENTRAL, and each WS send messages or irrigation requests at any time.

Technical design

Students are asked to implement a distributed system made up of, at least, the following software components:

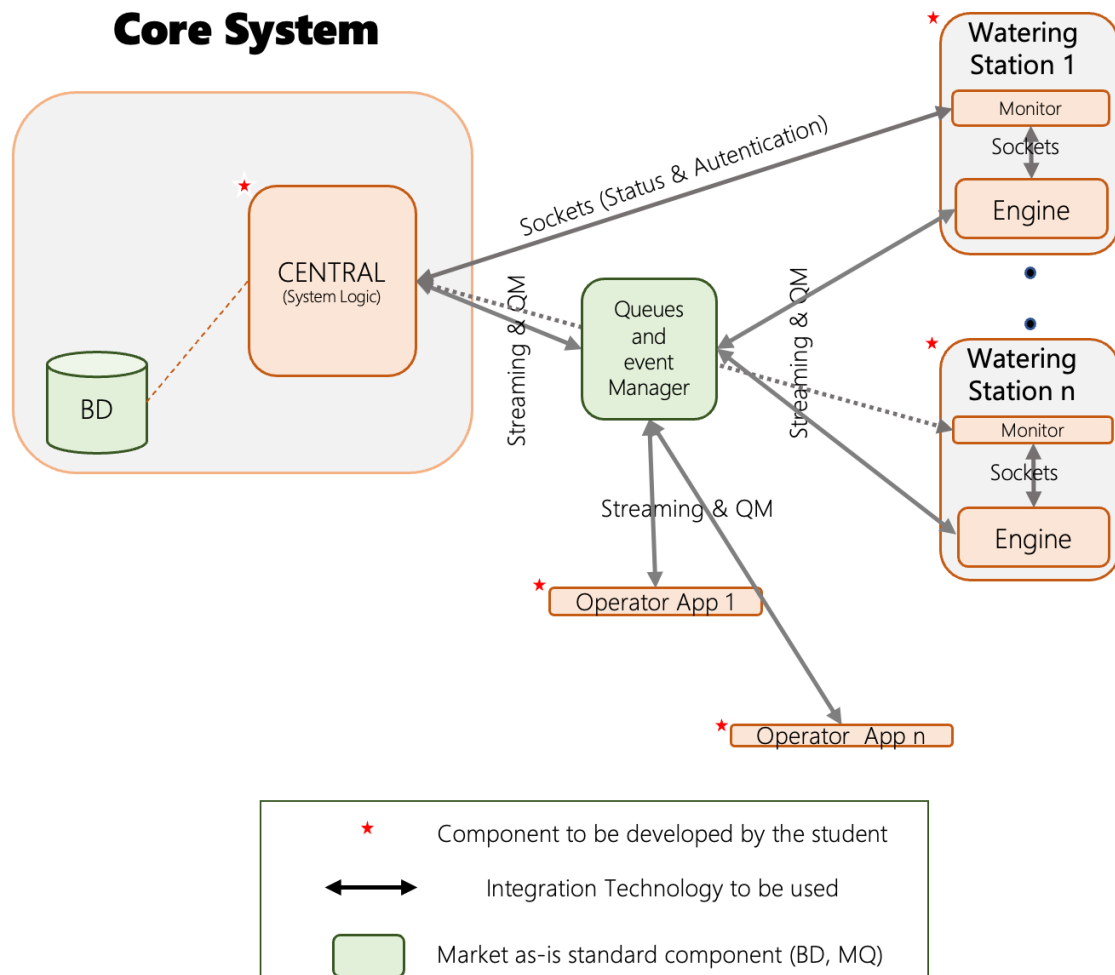


Figure 3. Conceptual architecture of the software system, interconnections between components, and interface types. Release 1

The software components the student must develop are the following:

- **Core System:** Core modules of the solution.
 - **CENTRAL:** Module representing the control center of the entire solution. It implements the logic and governance of the whole system.
- **Watering Station (WS):** Modules that simulate the irrigation stations.
 - **Engine:** Module that receives information from the flow meter and activates the solenoid valve.
 - **Monitor:** Module that monitors the health of the entire system and reports any fault (leak) to CENTRAL. It is also used to authenticate and register the WS with the central system when appropriate.
- **Operator App:** Application used by operators (gardeners) to control the irrigation stations.

The components can be developed in the student's language of preference: Java, C/C++, .NET, Python, etc., with the student assuming responsibility for their knowledge of it and how to deploy it.

The event streaming management subsystem will be implemented using KAFKA technology, whose operation will be explained during the lab sessions.

Each component is specified in more detail below.

Core System

Contains all the modules that implement the system's main structure.

CENTRAL:

This is the application that will implement the system's core logic. The application name will be **mandatorily "WM_Central"**. To run, it will receive at least the following command-line arguments:

- Listening port of the socket server it offers to WS_WM_M.
- IP and port of the queue manager's Broker/Bootstrap server.

When the application starts, it will listen on the configured port and implement all the functionality needed to run the solution mechanics described in the previous section **including those indicated in point 11** of the "Solution mechanics" section.

Database:

It will contain the operators' and WSs' data (ID, status, etc.) as well as any other information students consider necessary for implementing the system.

Students must mandatorily use SQLite for its implementation.

Watering Station (WS)

Engine:

This is the application that will implement the main logic of the whole system. The application name will be **mandatorily "WM_WS_E"**. To run, it will receive at least the following command-line arguments:

- IP and port of the queue manager's Broker/Bootstrap server.
- IP and port of the WS's Monitor

The application will remain waiting until it receives a service request from the central system. **The WS will be assigned the ID received in the WM_WS_M's command-line parameters, whose validity must be confirmed by WM_Central during the authentication process.**

Monitor:

Application simulating a monitoring module for the entire WS (hardware and software), ensuring its safety and correct incident-free operation. The application name will be **mandatorily** “WM_WS_M”. To run, it will receive at least the following command-line arguments:

- Port of the socket server it offers to WM_WS_E
- IP and port of WM_Central
- WS ID: unique identifier under which that WS is registered on the network.

When the application starts, it will connect to WM_Central to authenticate and confirm that the WS is operational and ready to provide service. Likewise, it will wait for the connection of the Engine of the WS it belongs to and will start **sending it a health-check message every second**. If no response is received from WM_WS_E, or a KO-type response is received, WM_WS_M will send WM_Central a fault message simulating a leak detection, as indicated in earlier sections. To simulate such incidents, the WM_WS_E application must allow a key to be pressed at runtime to report a KO to the monitor.

Field Operators (FO)

Application available to system users for requesting a supply. The application name will be **mandatorily** “WM_FO”. To run, it will receive at least the following command-line arguments:

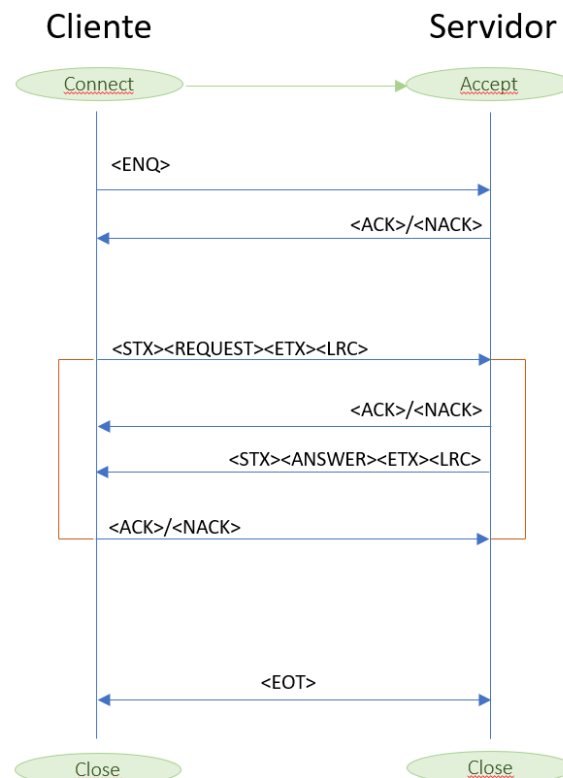
- IP and port of the queue manager's Broker/Bootstrap server.
- Operator ID: unique identifier under which that operator is registered in CENTRAL.

In addition to being able to request a one-off supply at a WS, this application will be able to read all the services it is going to request from a file. It will send the first one to WM_Central and, after it concludes (whether successfully or not), it will wait 4 seconds and move on to requesting the next service.

Communication protocols

Standard socket protocol

As mentioned at the beginning of this document, socket-based communication requires the parties involved to precisely define the messaging and protocol. Students may define the communication protocols between the system's different modules at their own discretion. **The instructors recommend, although it is not mandatory,** using messaging based on the <STX><DATA><ETX><LRC> packaging standard, which corresponds to the following flow:



Where:

- <REQUEST> and <ANSWER>: Contain the message transmitted between both endpoints, with the fields separated by a given character:
 - o E.g.: <REQUEST>: Operation Code#field1#...#field n
- <LRC>: Defined as the byte-by-byte XOR(MESSAGE), and used to verify that the message transmission was completed successfully and was correctly received by the recipient, who will respond with an <ACK> or <NACK>

Final remarks

In the face of functional inconsistencies or aspects not considered in the previous sections that prevent the correct implementation of the practical, students must implement the

alternative they consider most appropriate after consulting with the instructor to agree on that implementation, avoiding an excessive level of complexity.

Minimum deployment guide

For the practical to be properly evaluated, it is necessary to verify that the requested distributed application is deployed in a genuinely distributed environment.

It must be deployed mandatorily as follows:

- Cloud systems (hosted on RAILWAY www.railway.com): WM_Central, DB, and KAFKA
- Lab: several WM_FO on one PC and several WM_WS on another PC.

At least the following scenario must be deployed.

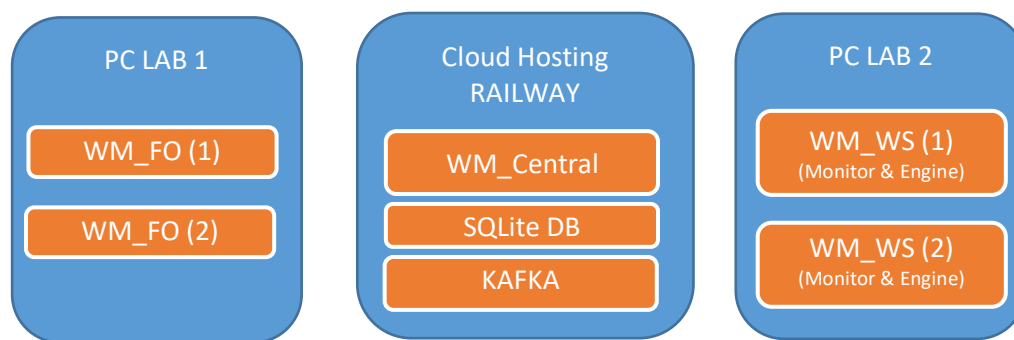


Figure 4. Minimum physical scenario for deploying the practical.

- The student must attend the evaluation with **the Cloud part fully deployed and running. Only the FO and WS modules will be deployed in the lab.**
- **The use of Docker technology for deployment is mandatory in all cases** for deploying any module.
- **The use of GitHub is mandatory for managing the code repository. Each group must give their lab instructor access to that repository.**
- **The CI/CD cycle must be: VS Code -> GitHub -> Railway (advisable for its simplicity) or VS Code -> GitHub -> AWS Learner Lab (more configuration required)**

Deliverables and evaluation

- The practical will be evaluated in the labs.
- **It may be done in groups of up to 2 people, without prejudice to the instructor being able, at the time of grading, to ask any student in the group about any aspect of any of the modules.**
- Students must deploy the practical that solves the above assignment themselves. They must deploy a complete system, that is, one central system, the WSs (engine and monitor), and Field Operator, all interconnected with each other. **This requirement is essential in order for grading to take place.**

- All sections that will appear in the Grading Guide provided for this purpose must be able to be evaluated positively or negatively.

On the use of Artificial Intelligence (AI) tools

- The use of AI assistants (such as ChatGPT, GitHub Copilot, or others) is allowed as a support tool during the development of the practical, the architecture design, and error debugging. However, **uncritical use of AI (copying and pasting code without understanding how it works) severely penalizes the learning process.**
- Keep in mind that everything implemented in this practical is directly aligned with the course's official syllabus: from **communication between distributed components (sockets)** and **event-based and microservices patterns**, to **scalability, resilience patterns**, and the aspects of **security**. In the practical defense and in the **theory exam** absolute mastery of these concepts will be required. A student who has simply delegated the code to an AI without understanding the message frames, concurrency, or architectural styles will not be able to pass the individual assessment tests or the associated theoretical questions.
- AI must be used exclusively as support for getting past blockers or speeding up repetitive tasks, it being the students' absolute responsibility to know, explain, and be able to modify any line of code in their repository without outside help.
- The prompts used with AI for developing it must be included in GitHub and shared with the teaching staff.

In addition, students must submit, via the UACloud evaluation feature, before the date set by their lab instructor, a **lab report**, with the generated source code and compiled binaries, as well as a document detailing the following information. The format is free, but it must be a well-organized, properly formatted document, with careful attention to writing and spelling.

- Cover page with the students' first and last names and ID numbers, the academic year, and the title of the practical.
- A report indicating the name of the software components developed and a description of each one, also explaining and submitting the source code for all of them.
- A step-by-step deployment guide for the application, which must be the same one used during the grading of the practical.
- Screenshots showing the different applications working while connected.

Each lab instructor may request any other evidence from students that the instructor deems appropriate in order to finalize the evaluation.

The submission deadline will fall in the week of October 26, 2026.